



International Islamic University Chittagong

Department of Computer Science and Engineering

Lab Report

Lab No 6: Implementation of the Bresenham's Circle Algorithm

Course Title: Computer Graphics Lab

Course Code: CSE-4742

Session: Autumn 2026

Semester: 7th

Section: 7BF

Student Name	Tahasina Tasnim Afra
Student ID	C233456
Department	Computer Science and Engineering

Submitted To

Mrs. Sultana Tasnim
Lecturer, Department of CSE, IIUC

Submission Date:

September 20, 2026

1. Number & Name of the Experiment

Lab 5: Implementation of Bresenham's Circle Algorithm

2. Introduction

Bresenham's Circle Algorithm is an incremental scan-conversion algorithm used to generate a circle on a raster display. Instead of using square-root, trigonometric, or floating-point calculations, the algorithm uses an integer decision parameter to determine the appropriate pixels for drawing the circle.

For a circle centered at $(0, 0)$ with radius r , the mathematical equation is

$$x^2 + y^2 = r^2.$$

The algorithm calculates the pixels for one octant of the circle and uses eight-way symmetry to generate the remaining points. The initial values are

$$x = 0, \quad y = r, \quad d = 3 - 2r.$$

At each step, the decision parameter determines whether only x is increased or both x is increased and y is decreased. The process continues until $x > y$.

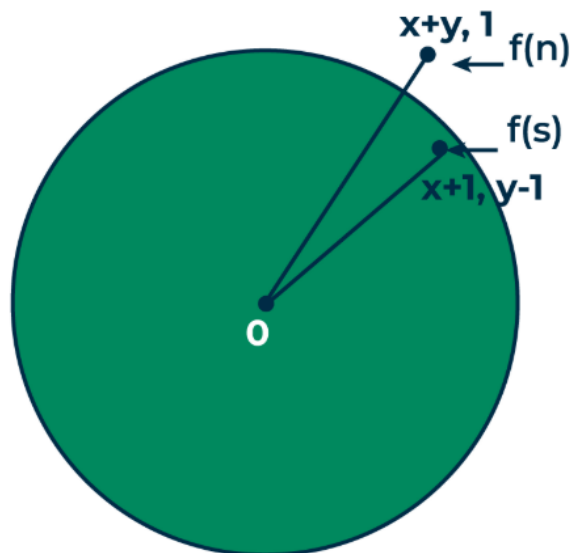


Figure 1: Bresenham's Circle Algorithm

3. Algorithm

Bresenham's Circle Algorithm starts from the top point of the circle and calculates the pixels from approximately 90° to 45° . The remaining points are generated using eight-way symmetry.

The initial values are

$$x = 0, \quad y = r, \quad d = 3 - 2r.$$

Initial Point

The algorithm starts from the top of the circle:

$$(x, y) = (0, r)$$

For the selected radius of 200, the initial point is

$$(0, 200).$$

The initial decision parameter is

$$d = 3 - 2r.$$

For $r = 200$,

$$d = 3 - 2(200) = -397.$$

Decision Parameter

At each iteration, the value of d determines the next pixel.

Case 1: $d < 0$

When

$$d < 0,$$

the value of y remains unchanged. The decision parameter is updated as

$$d = d + 4x + 6.$$

After the update, x is increased by one:

$$x = x + 1.$$

Case 2: $d \geq 0$

When

$$d \geq 0,$$

both x is increased and y is decreased. The decision parameter is updated as

$$d = d + 4(x - y) + 10$$

and

$$y = y - 1.$$

After this, x is increased by one:

$$x = x + 1.$$

Eight-Way Symmetry

After calculating a point (x, y) , the remaining seven points are obtained using eight-way symmetry. This is possible because a circle has symmetry about the horizontal, vertical, and diagonal axes.

For a circle centered at (h, k) , the eight symmetric points are

$$\begin{array}{ll} (h + x, k + y), & (h + y, k + x), \\ (h - y, k + x), & (h - x, k + y), \\ (h - x, k - y), & (h - y, k - x), \\ (h + y, k - x), & (h + x, k - y). \end{array}$$

Stopping Condition

The algorithm calculates only one octant of the circle. The loop continues while

$$x \leq y.$$

When $x > y$, the algorithm stops because the remaining points can already be obtained through eight-way symmetry. At approximately the 45° position,

$$x \approx y \approx \frac{r}{\sqrt{2}}.$$

For $r = 200$, the calculated points continue up to approximately $(141, 142)$.

Implementation

The implementation follows the incremental decision-parameter method. The `BresenhamCircle()` function initializes x , y , and d , then determines the next pixel while $x \leq y$. If $d < 0$, the decision parameter is updated using $d = d + 4x + 6$. Otherwise, it is updated using $d = d + 4(x - y) + 10$ and y is decreased by one. In both cases, x is then increased by one.

The program uses `plotCirclePoints()` to generate the eight symmetric points around the circle center using `putpixel()`. The calculated (x, y) coordinates are also displayed using `cout`, allowing the generated points to be observed during execution.

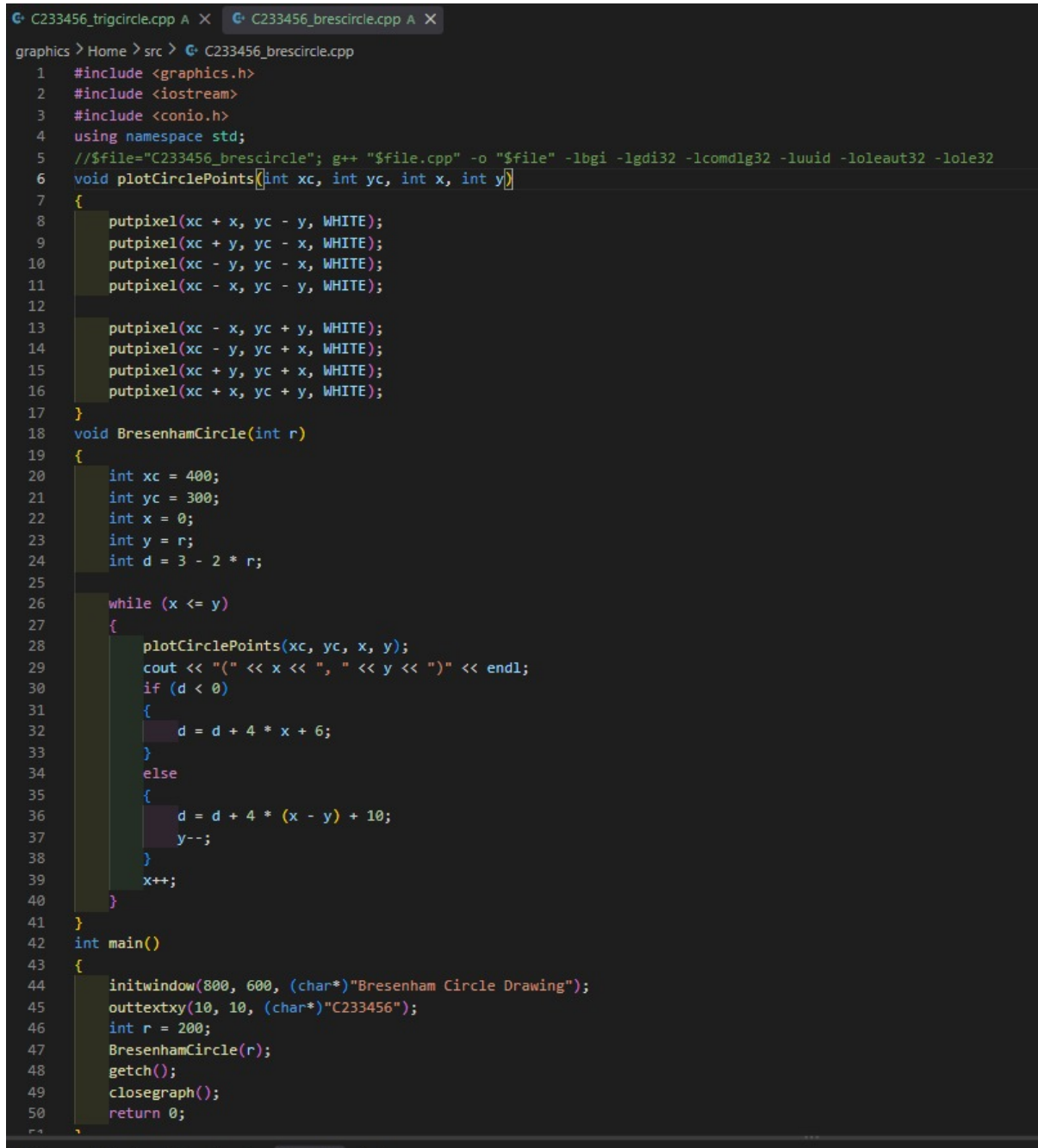
4. Program with Screenshot

Code

The C++ implementation of Bresenham's Circle Algorithm is available in the `AFRA-com-p-graphics` GitHub repository. The code used for the experiment is shown below.

Code Screenshot

The following screenshot shows the C++ implementation of Bresenham's Circle Algorithm:



```

1  #include <graphics.h>
2  #include <iostream>
3  #include <conio.h>
4  using namespace std;
5  // $file="C233456_brescircle"; g++ "$file.cpp" -o "$file" -lbgi -lgi32 -lcomdlg32 -luuid -oleaut32 -ole32
6  void plotCirclePoints(int xc, int yc, int x, int y)
7  {
8      putpixel(xc + x, yc - y, WHITE);
9      putpixel(xc + y, yc - x, WHITE);
10     putpixel(xc - y, yc - x, WHITE);
11     putpixel(xc - x, yc - y, WHITE);
12
13     putpixel(xc - x, yc + y, WHITE);
14     putpixel(xc - y, yc + x, WHITE);
15     putpixel(xc + y, yc + x, WHITE);
16     putpixel(xc + x, yc + y, WHITE);
17 }
18 void BresenhamCircle(int r)
19 {
20     int xc = 400;
21     int yc = 300;
22     int x = 0;
23     int y = r;
24     int d = 3 - 2 * r;
25
26     while (x <= y)
27     {
28         plotCirclePoints(xc, yc, x, y);
29         cout << "(" << x << ", " << y << ")" << endl;
30         if (d < 0)
31         {
32             d = d + 4 * x + 6;
33         }
34         else
35         {
36             d = d + 4 * (x - y) + 10;
37             y--;
38         }
39         x++;
40     }
41 }
42 int main()
43 {
44     initwindow(800, 600, (char*)"Bresenham Circle Drawing");
45     outtextxy(10, 10, (char*)"C233456");
46     int r = 200;
47     BresenhamCircle(r);
48     getch();
49     closegraph();
50     return 0;

```

Figure 2: C++ implementation of Bresenham's Circle Algorithm

5. Output/Result

The program was successfully executed and generated a complete circle using Bresenham's Circle Algorithm. The algorithm calculated the points for one portion of the circle using the integer decision parameter and used eight-way symmetry to obtain the remaining points.

For the selected radius of 200, the calculation started from the initial point (0, 200) and continued until the condition $x \leq y$ was no longer satisfied. The calculated points were

plotted using `putpixel()`.

The student ID (C233456) was also displayed in the graphics window, and the complete circle was successfully generated.

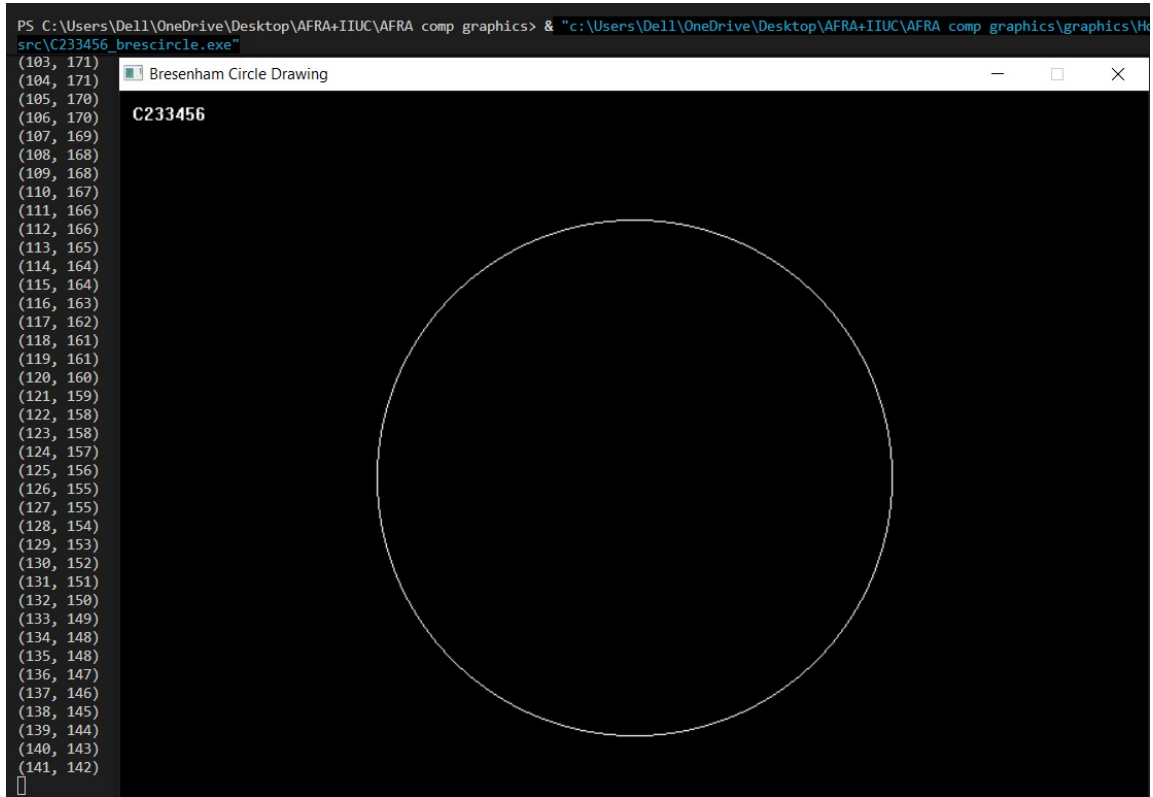


Figure 3: Output of Bresenham's Circle Algorithm for radius $r = 200$

6. Discussion

Bresenham's Circle Algorithm successfully generated a circle using incremental integer calculations and a decision parameter. The algorithm started with $x = 0$, $y = r$, and $d = 3 - 2r$.

At each iteration, the value of the decision parameter was used to select the next pixel. When $d < 0$, the decision parameter was updated using

$$d = d + 4x + 6.$$

When $d \geq 0$, it was updated using

$$d = d + 4(x - y) + 10$$

and the value of y was decreased by one. The value of x was then increased by one in both cases.

Only one octant of the circle was calculated directly, while eight-way symmetry was used to generate the complete circle. This reduces the number of points that need to be calculated directly and makes the algorithm suitable for raster graphics.

For the selected radius of 200, the calculated points started from $(0, 200)$ and continued to approximately the 45° position. The algorithm has a time complexity of $O(r)$ and a space complexity of $O(1)$.

7. Conclusion

Bresenham's Circle Algorithm was successfully implemented in C++ using the `graphics.h` library. The experiment demonstrated how a circle can be scan converted into pixels using an incremental integer decision parameter instead of directly using square-root or trigonometric calculations.

The algorithm started with $x = 0$, $y = r$, and the initial decision parameter $d = 3 - 2r$. At each step, the value of d was used to determine the appropriate pixel and update the values of x and y . Only one octant of the circle was calculated directly, while eight-way symmetry was used to generate the remaining points.

The experiment also demonstrated how integer-based incremental calculations can be used for efficient circle rasterization. The calculated points were mapped to the graphics window and displayed using `putpixel()`. Overall, the implementation provided a clear understanding of Bresenham's decision-parameter method, circle scan conversion, eight-way symmetry, and pixel plotting.